

Cầu thang cho các thí sinh olympic

Giới hạn thời gian: 0.5 giây

Giới hạn bộ nhớ: 1024 MB

Tại Trung tâm Giáo dục “Sirius”, những bậc thang là nơi yêu thích để học sinh tập trung và giao lưu không chính thức. Tuy nhiên, số lượng thí sinh tham dự olympic tin học vượt xa số lượng học sinh của bất kỳ chương trình giáo dục nào khác, và trong số các cầu thang có sẵn thì không tìm được cầu thang nào phù hợp. Vì vậy, bộ phận trang bị quyết định xây dựng một chiếc cầu thang mới, sử dụng một bản thiết kế đặc biệt.

Bản thiết kế được cho dưới dạng một bảng gồm h hàng và w cột, được đánh số từ trên xuống và từ trái sang phải. Trong mỗi ô của bảng có ghi một số — 0 hoặc 1. Cầu thang chỉ có thể được tạo ra từ các ô có chứa 1.

Một chiếc cầu thang được hình thành từ một tập hợp các ô có chứa 1, nằm trong một số hàng liên tiếp của bảng. Trong mỗi hàng thuộc cầu thang, tập hợp các ô được chọn phải tạo thành một đoạn liên tục. Đồng thời, ở mỗi hàng tiếp theo (tính từ trên xuống) của cầu thang, số lượng ô được chọn phải không ít hơn số lượng ô được chọn ở hàng ngay phía trên nó, và các ô ngoài cùng bên trái trong mỗi hàng phải nằm cùng một cột.

Dưới đây là một ví dụ về một chiếc cầu thang:

1		
1	1	
1	1	
1	1	1



Yêu cầu

Hãy tìm trong bảng đã cho số lượng ô lớn nhất có thể tạo thành một chiếc cầu thang.

Định dạng dữ liệu vào từ file văn bản “stairs.inp”

- Dòng đầu tiên chứa hai số nguyên h và w
($1 \leq h, w \leq 2 \cdot 10^5, h \cdot w \leq 4 \cdot 10^6$) — số hàng và số cột của bảng.
- Mỗi trong số h dòng tiếp theo chứa w ký tự 0 hoặc 1 — chính là các giá trị trong các ô của bảng.

Định dạng dữ liệu ra file văn bản “stairs.out”

Xuất ra một số nguyên — số lượng ô lớn nhất tạo thành cầu thang.

Hệ thống chấm điểm

Bài toán con	Điểm	Giới hạn thêm	Bài toán con cần thiết
		h, w	
1	25	$h, w \leq 50$	
2	25	$h, w \leq 400$	1
3	25	$h \cdot w \leq 200000$	1, 2
4	25		1-3

Ví dụ:

stairs.inp	stairs.out
6 4 0011 1101 0111 1110 0111 0100	8

Ghi chú

Hình dưới đây minh họa cho ví dụ đầu tiên. Cầu thang, được tạo thành từ số lượng ô tối đa có thể trong bảng, được tô màu xám.

0	0	1	1
1	1	0	1
0	1	1	1
1	1	1	0
0	1	1	1
0	1	0	0

Chuyển ca ở Sirius

Giới hạn thời gian: 1 giây

Giới hạn bộ nhớ: 1024 megabyte

Các học viên của những chương trình giáo dục đôi khi thắc mắc vì sao giữa hai chương trình thường có quãng nghỉ vài ngày. Lý do rất đơn giản: sau mỗi chương trình, nhân viên của Sirius cần thời gian để dọn dẹp, chỉnh trang lại các phòng ở.

Trên một tầng của khách sạn thuộc “Sirius” có n phòng, được đánh số từ 1 đến n . Sau khi kết thúc chương trình, tất cả các phòng này đều cần sửa chữa.

Có k nhân viên, được đánh số từ 1 đến k , tham gia công việc sửa chữa. Với nhân viên thứ i gán một đoạn phòng $[l_i, r_i]$ (bao gồm cả hai đầu), và một số m_i thuộc đoạn đó — đây là phòng mà nhân viên sẽ bắt đầu đi kiểm tra dây phòng của mình. Các đoạn của những nhân viên khác nhau có thể giao nhau hoặc thậm chí trùng h lẫn nhau.

Các nhân viên được cử đi từ căn cứ theo một thứ tự nào đó; chỉ khi người trước đó trở về căn cứ thì người tiếp theo mới được cử đi.

Khi cử nhân viên thứ i đi làm, trước hết anh/chị ấy sẽ đến phòng m_i .

- Nếu phòng này vẫn cần sửa, nhân viên sẽ sửa phòng đó, đồng thời đi qua toàn bộ các phòng trong đoạn $[l_i, r_i]$ mình phụ trách và sửa tất cả các phòng còn cần sửa trong đoạn này, rồi quay về căn cứ. Sau đó, toàn bộ các phòng trong đoạn $[l_i, r_i]$ sẽ không còn cần sửa nữa.

- Nếu phòng đầu tiên ghé qua m_i đã không còn cần sửa (do đồng nghiệp trước đó đã sửa), thì nhân viên lập tức quay về căn cứ, hy vọng rằng các phòng còn lại trong đoạn $[l_i, r_i]$ cũng đã được sửa hết. Trong trường hợp này, một số phòng khác trong đoạn $[l_i, r_i]$ vẫn có thể còn cần sửa.

Hãy xác định xem, với cách vận hành như trên, có thể sắp xếp thứ tự cử các nhân viên sao cho cuối cùng tất cả các phòng từ 1 đến n đều được sửa hay không.

Dữ liệu vào từ file văn bản “repair.inp”

Mỗi test gồm nhiều bộ dữ liệu.

Dòng đầu chứa một số nguyên t ($1 \leq t \leq 10^5$) — số bộ dữ liệu. Tiếp theo là mô tả từng bộ.

Dòng đầu của mỗi bộ dữ liệu chứa hai số nguyên n và k ($1 \leq n, k \leq 5 \cdot 10^5$) — số phòng và số nhân viên.

Mỗi trong k dòng tiếp theo chứa ba số nguyên l_i, m_i, r_i ($1 \leq l_i \leq m_i \leq r_i \leq n$) — phòng đầu của đoạn phụ trách, phòng xuất phát m_i trong đoạn đó, và phòng cuối của đoạn phụ trách của nhân viên thứ i .

Bảo đảm rằng tổng các giá trị n và k qua tất cả các bộ dữ liệu không vượt quá $5 \cdot 10^5$.

Dữ liệu ra file văn bản “repair.out”

Với mỗi bộ dữ liệu, in ra trên một dòng: “YES” nếu có thể sửa hết tất cả các phòng, và “NO” nếu không thể.

Hệ thống chấm điểm

Ký hiệu N là tổng của n qua tất cả các bộ dữ liệu, K là tổng của k qua tất cả các bộ dữ liệu.

Bài toán con	Điểm	Giới hạn thêm			Bài toán con cần thiết
		n, N	k, K	thêm	
1	5	-	$K \leq 10^4$	$m_i = l_i$	
2	5	$N \leq 500$	$k \leq 8$		
3	2	$n \leq 18$	$K \leq 500$		
4	12	$n \leq 50$	$K \leq 50$		
5	9	$N \leq 500$	$K \leq 500$		4
6	8	$n \leq 150$	$K \leq 500$		
7	6	-	$K \leq 10^4$	Mỗi nhân viên được gán với phòng số 1 hoặc phòng số n	
8	18	-	$K \leq 10^4$	Đối với mỗi nhân viên sẽ có một phòng được gán riêng chỉ cho anh/chị ấy	
9	3	-	-	Đối với mỗi nhân viên sẽ có một phòng được gán riêng chỉ cho anh/chị ấy	8
10	4	-	$K \leq 10^4$	$r_i - l_i = r_j - l_j$ với $\forall i, j$	
11	4	-	$K \leq 10^4$	Bất kỳ m_i trùng với l_i hoặc r_i	1
12	4	$n \leq 10^4$	$K \leq 10^4$		2-6
13	6	-	$K \leq 10^4$		1-8,10-12
14	14	-	-		1-13

Ví dụ:

repair.inp	repair.out
2	YES
5 2	NO
3 4 5	
1 3 3	
5 3	
1 2 4	
2 4 5	
3 3 3	

Ghi chú

Trong **bộ dữ liệu đầu tiên** của ví dụ:

- Trước hết cần cử **nhân viên thứ hai** đi sửa, anh ta sẽ sửa các phòng từ số 1 đến số 3.
- Sau đó **nhân viên thứ nhất** được cử đến phòng số 4. Vì phòng này vẫn còn cần sửa nên nhân viên thứ nhất sẽ tiếp tục sửa tất cả các phòng còn lại trong đoạn mà anh ta phụ trách.
- Kết quả: toàn bộ các phòng đều được sửa xong.

Trong **bộ dữ liệu thứ hai**, **không thể** chọn một thứ tự cử nhân viên sao cho tất cả các phòng đều được sửa.

Công viên Sochi

Giới hạn thời gian: 1 giây

Giới hạn bộ nhớ: 1024 megabyte



Tại Công viên Sochi vừa khai trương một trò chơi mới. Trên một đường thẳng có n mục tiêu; tọa độ của mục tiêu thứ i là x_i ($1 \leq i \leq n$). Khách tham quan phải bắn trúng tất cả các mục tiêu này theo bất kỳ thứ tự nào.

Để bắn mục tiêu, người chơi dùng bóng. Nếu người chơi đứng tại điểm có tọa độ x và muốn bắn vào mục tiêu ở x_i , họ sẽ tốn $(x - x_i)^2$ calo.

Người chơi bước vào khu trò chơi tại điểm x_0 . Nguồn bóng không giới hạn đặt tại điểm vào x_0 , và tại mọi điểm cách nhau d đơn vị, tức các điểm $x_0 + k \cdot d$ với k là số nguyên bất kỳ. Theo quy định của trò chơi, không được mang bóng theo, vì vậy chỉ được ném bóng từ các điểm nói trên.

Trong ngày nghỉ giữa các lượt thi, có m thí sinh Olympic ghé thăm Công viên Sochi. Thể lực mỗi người khác nhau, nên với thí sinh thứ j , để di chuyển một quãng d tốn t_j calo. Bạn cần xác định lượng calo tối thiểu cần cho mỗi thí sinh để có thể di chuyển và bắn trúng tất cả các mục tiêu của trò chơi.

Dữ liệu vào từ file văn bản “park.inp”

- Dòng 1: một số nguyên n ($1 \leq n \leq 3 \cdot 10^5$) — số lượng mục tiêu.
- Dòng 2: n số nguyên $x_1, x_2, \dots, x_n$ ($0 \leq x_i \leq 10^9$) — tọa độ các mục tiêu.
- Dòng 3: hai số nguyên x_0 và d ($0 \leq x_0 \leq 10^9, 1 \leq d \leq 2 \cdot 10^6$) — điểm vào của người chơi và khoảng cách giữa các điểm có kho bóng.
- Dòng 4: một số nguyên m ($1 \leq m \leq 6 \cdot 10^5$) — số thí sinh.
- Tiếp theo m dòng, mỗi dòng một số nguyên t_j ($0 \leq t_j \leq 10^8$) — năng lượng cần thiết cho thí sinh thứ j để di chuyển giữa hai điểm kho bóng kề nhau.

Dữ liệu ra file văn bản “park.out”

Với mỗi thí sinh, in ra một số nguyên — lượng calo tối thiểu cần thiết để di chuyển và bắt trúng tất cả các mục tiêu.

Với các ràng buộc đã cho, đáp án không vượt quá giá trị lớn nhất của kiểu số nguyên có dấu 64-bit. Tuy nhiên, trong quá trình tính toán trung gian có thể cần dùng kiểu `__int128` trong C++ (chỉ được hỗ trợ bởi trình biên dịch GNU C++), `BigInteger` trong Java, hoặc `int` trong Python.

Hệ thống chấm điểm

Bài toán con	Điểm	Giới hạn thêm				Bài toán con cần thiết
		n	x_0	m	thêm	
1	9	-	-	$m = 1$	$t_1 = 0$	
2	7	$n = 1$	-	$m \leq 10^4$	-	
3	8	$n = 2$	-	$m \leq 10^4$	$x_1 \leq x_0 \leq x_2$	
4	3	$n \leq 50$	$x_0 = 0$	$m \leq 50$	$d \leq 50, x_i \leq 50$	
5	2	$n \leq 50$	$x_0 \leq 50$	$m \leq 50$	$d \leq 50, x_i \leq 50$	
6	4	-	$x_0 = 0$	$m \leq 10$	$x_i \leq 10^6$	
7	2	-	$x_0 \leq 10^6$	$m \leq 10$	$x_i \leq 10^6$	
8	6	-	$x_0 = 0$	$m \leq 10^4$	$x_i \leq 10^6, x_i \leq 10^6$	
9	10	-	$x_0 \leq 10^6$	$m \leq 10^4$	$x_i \leq 10^6$	
10	7	-	$x_0 \leq 10^6$	$m \leq 10^5$	$x_i \leq 10^6$	
11	2	-	-	$m \leq 10$	-	
12	12	-	$x_0 = 0$	$m \leq 10^5$	$d = 1$	
13	5	-	-	$m \leq 10^5$	$d = 1$	
14	8	-	$x_0 = 0$	$m \leq 10^5$	-	
15	2	-	-	$m \leq 10^5$	-	
16	1	-	-	$m \leq 2 \cdot 10^5$	-	
17	3	-	-	$m \leq 3 \cdot 10^5$	-	
18	9	-	-	-	-	

Ví dụ:

park.inp	park.out
3	3
4 0 7	7
2 3	10
7	12
0	13
1	32
2	33
3	
4	
23	
25	

4 30 239 57 179 0 7 5 1 10 15 100 100000	49 355 525 3378 93311
4 100 2 101 666 9 10 5 777 1 2 15 10	49597 91 159 1043 703

Ghi chú:

Trong test thứ nhất, với người tham gia thứ hai ($t_2 = 1$), chiến lược tối ưu để bắn các mục tiêu là:

1. Di chuyển từ điểm vào $x_0 = 2$ đến điểm $x_0 - d = -1$, tốn $t_2 = 1$ calo. Lưu ý: tọa độ của người chơi có thể âm.
2. Bắn mục tiêu tại $x_2 = 0$, tốn $(-1 - 0)^2 = 1$ calo.
3. Di chuyển đến điểm $-1 + 2d = 5$, tốn $2 \cdot t_2 = 2$ calo.
4. Bắn mục tiêu tại $x_1 = 4$, tốn $(5 - 4)^2 = 1$ calo.
5. Di chuyển đến điểm $5 + d = 8$, tốn $t_2 = 1$ calo.
6. Bắn mục tiêu tại $x_3 = 7$, tốn $(8 - 7)^2 = 1$ calo.

Tổng năng lượng tiêu hao là:

$$1 + 1 + 2 + 1 + 1 + 1 = 7 \text{ calo.}$$

Có thể chứng minh rằng đây là lượng năng lượng tối thiểu.

Với người tham gia thứ sáu ($t_6 = 23$), chiến lược tối ưu là:

1. Bắn mục tiêu tại $x_2 = 0$, tốn $(2 - 0)^2 = 4$ calo.
2. Di chuyển đến điểm $2 + d = 5$, tốn $t_6 = 23$ calo.
3. Bắn mục tiêu tại $x_3 = 7$, tốn $(7 - 5)^2 = 4$ calo.
4. Bắn mục tiêu tại $x_1 = 4$, tốn $(5 - 4)^2 = 1$ calo.

Tổng năng lượng tiêu hao là:

$$4 + 23 + 4 + 1 = 32 \text{ calo.}$$

Đây cũng là lượng năng lượng tối thiểu có thể đạt được.